# AI-Driven Agentic Smart Event Ticket Booking Platform

Dr. Neelima Gogineni
Assistant Professor
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation, Bowrampet
Hyderabad-500043, Telangana, India
Email: dr.neelima@klh.edu.in

K.Yaswanth Reddy
Department of Computer Science and Information Technology
Koneru Lakshmaiah Education Foundation, Bowrampet
Hyderabad-500043, Telangana, India
Email: 2420090053@klh.edu.in

U. Chaithanya Karthik
Department of Computer Science and Information Technology
Koneru Lakshmaiah Education Foundation, Bowrampet
Hyderabad-500043, Telangana, India
Email: 2420090030@klh.edu.in

V. Sudhir
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation, Bowrampet
Hyderabad-500043, Telangana, India
Email: 2420030377@klh.edu.in

***Abstract*—The growing popularity of concerts, live music performances, DJ shows, cultural festivals, and techno events has The need to easily search for and purchase tickets to an event has increased. Users will need to search through many listings to find events that are relevant to them based on their interests, location, budget and desired date(s). This paper presents EventGenie, an AI-Driven Agentic Smart Event Ticket Booking Platform intended to combine event discovery, contextual recommendations, and guided booking in one event-focused application. The proposed architecture uses React for the frontend and Spring Boot backend components, with separate responsibilities for user management, event management, request routing, recommendation, and booking. Its recommendation approach considers event category, location, budget, and timing, while a proposed agentic workflow interprets natural-language requests and guides users toward suitable events. This paper reviews relevant event-recommendation research, presents the proposed architecture and ranking model, and defines an evaluation plan for functional correctness, recommendation quality, and response time. Quantitative performance claims are reserved until the completed EventGenie implementation has been tested using reproducible experiments.**

***Index Terms*—EventGenie, event ticket booking, event recommendation, contextual recommendation, agentic systems, Spring Boot, React, microservices.**

## I. INTRODUCTION

There has been a change in the way people find and get involved with entertainment and culture using digital media. Concerts, live Bands, DJ Shows, Cultural Programs and Techno Festivals attract a wide variety of audiences to attend. As there are more events to choose from there needs to be an easy way to find events. way to identify suitable events and complete ticket bookings.

Conventional booking interfaces commonly provide event listings, search facilities, and category filters. find an event that suits them. However, there could be a lot of listing to go through in order to find events that suit their interests, location, date and budget.. Event recommendation is a distinct problem because events are temporary, availability changes over time, and preferences may depend on context [1], [3], [4].

This paper presents EventGenie, an AI-Driven Agentic Smart Event Ticket Booking Platform focused on live entertainment and cultural events in India, initially emphasizing Hyderabad and nearby venues. The platform is designed to provide users with: event search, contextual recommendations and guided booking in one application. Its intended architecture separates user management, event management, request routing, and future recommendation and booking functionality.

The contributions are: (1) an event-focused design for live entertainment and cultural activities; (2) a contextual ranking model using category, location, budget, and timing; (3) a modular backend design with separate service responsibilities; (4) a proposed agentic workflow for natural-language discovery and guided booking; and (5) an evaluation plan with measurable tests.

## II. PROBLEM STATEMENT AND OBJECTIVES

Users can search through a large number of events to find ones which fit their interests, location, dates and budget. Filtering by category alone may not give specific results.. A practical booking platform must connect discovery with event information, booking validation, and clear confirmation.

The objectives are to organize events by category and venue; rank events according to preferences and context; separate user and event responsibilities; guide users from search to booking confirmation; and evaluate the implementation through reproducible tests.

This resulted in the following research questions. First, how can a focused event catalogue reduce the effort needed to find a suitable live event? Second, which contextual attributes can be combined in a transparent ranking baseline without requiring a large training dataset? Third, how can a service-oriented

architecture separate event discovery from user-related operations while retaining a coherent user experience? Finally, what tests are required before the platform can claim that recommendations or booking workflows are reliable?

The scope of this paper is deliberately limited to an academic prototype. It discusses the system structure and a candidate recommendation method, but does not claim that an autonomous agent, trained recommendation model, production payment gateway, or large-scale deployment is already available. This distinction is important because a design proposal and a validated implementation provide different types of evidence.

## III. Literature Survey

### A. Event Recommendation

Qiao *et al.* studied event recommendation in event-based social networks using heterogeneous social relationships and implicit feedback [1]. Arens-Volland and Naudet described a personalized event recommender using semantic enrichment and professional social-network information [2]. Horowitz *et al.* presented EventAware, a mobile event recommender combining tag-based and context-aware techniques [3]. These studies highlight the importance of user information and event context.

A recurring issue in event recommendation is that events differ from persistent products. A product can remain available for months or years, whereas a concert or cultural programme has a fixed date and venue. A recommendation that ignores event timing may be relevant in subject matter but unusable in practice. Furthermore, an event occurring far away from a users location could be appealing but very impractical due to travel time and costs.Therefore, context should also provide information to aid in the ranking of preferences but also to eliminate events that violate explicit constraints.

Collaborative approaches can learn from interactions among users, but they depend on sufficient interaction history. A newly developed platform may have a cold-start problem: new users have not made bookings and new events have no click or booking history. Content-based and rule-based ranking can provide a transparent initial baseline because they can use attributes already present in an event listing. As interaction data accumulates, hybrid methods may combine content attributes with user behaviour, subject to consent and data-quality safeguards.

### B. Location and Context

Jiménez, Salamó, and Boratto investigated distance-aware event recommendation and considered the distance between users and events [4]. Wei and Wang proposed a contextual and personalized event recommender that considers content preferences and temporal, spatial, and cost-related factors [5]. Such factors are relevant when event choice depends on venue, date, and budget.

General recommender-system research also distinguishes content-based, collaborate, knowledge-based, and hybrid approaches [6], [7].based on what data you have available. For EventGenie, an interpretable weighted score is suitable as a baseline because its components can be inspected and tested individually. A future implementation could compare this baseline against category-only filtering and a hybrid method, using the same candidate events and a documented evaluation set.

Table I summarizes how selected literature relates to the design decisions in this work. The table is a qualitative synthesis of the cited publications; it is not a claim that the cited systems were experimentally compared with EventGenie.

### C. C. Research Gap

Previous works have shown the importance of context, location,user preferences in event recommendation. EventGenie focuses on connecting these factors with a practical ticket-booking workflow and a modular application architecture. Its proposed agentic component aims to assist users from natural-language search through selection and booking guidance. Novelty and effectiveness must be assessed through implementation and comparative evaluation, rather than assumed from the design.

## IV. Proposed System

### A. Architecture

The current project work has focused on a React frontend and Spring Boot service-based backend, including an API Gateway, Eureka service discovery, User Service, and Event Service. The Event Service is intended to expose event information through REST endpoints, while the User Service handles user-related operations. EventGenie is focused on Indian live events such as concerts, live bands, DJ shows, cultural festivals, and techno events, with Hyderabad venues among the initial examples. Personalized recommendations and a separate persistent booking workflow are shown as planned extensions in Fig. 1; they must not be interpreted as completed services unless confirmed by the repository and tests.

### B. Event Discovery

he platform helps to manage concerts, live bands,DJ, cultural festivals, techno events. Each event can be listed with name type date location price description available Searching and filtering can be performed to assist narrowing down of options.

An event entry must have a consistent identifier for the frontend can refer to a listing consistently. Typical fields include a title, description, event category, venue name, city, date and start time, ticket price or price range, and availability status. Required fields should be validated before a listing is displayed. If an attribute is unknown, the interface should avoid presenting a guessed value. Event dates and prices are especially important because stale information can lead to a poor recommendation or a failed booking attempt.

A focused catalogue also benefits from normalized category labels. For example, a cultural festival should not be classified inconsistently as both a concert and a general entertainment

TABLE I
SELECTED LITERATURE AND ITS RELEVANCE TO EVENTGENIE

| Study | Main focus reported | Design implication | Difference in this work |
|---|---|---|---|
| Qiao et al. [1] | Event recommendation in event-based social networks | Use event context and interaction information where available | Focuses on a ticket-booking application and explicit constraints |
| Arens-Volland and Naudet [2] | Personalized recommendations for event attendees | Personalization can use enriched user and event information | Proposes a simpler baseline suited to an early prototype |
| Horowitz et al. [3] | Mobile, tag-based and context-aware event recommendations | Context and event tags can assist event discovery | Connects event discovery to service-based application design |
| Jiménez et al. [4] | Distance-aware event recommendation | Location compatibility should be represented explicitly | Treats location as one factor alongside budget and timing |
| Wei and Wang [5] | Contextual and personalized event recommendation | Consider spatial, temporal, and cost-related context | Defines an application-oriented ranking baseline for future validation |
| Adomavicius and Tuzhilin [6]; Burke [7] | Recommender-system approaches and hybrid methods | Select a method according to available data and evaluation needs | Establishes a baseline before considering hybrid personalization |

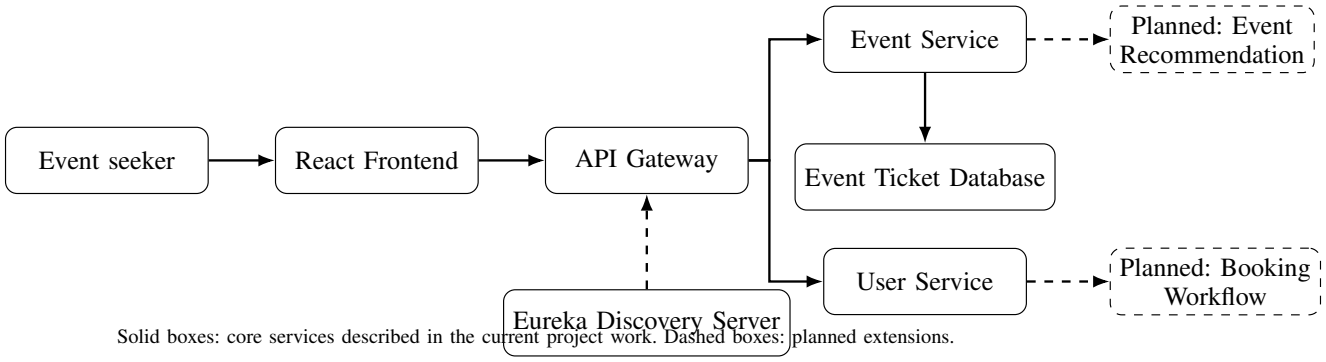



Fig. 1. EventGenie architecture aligned with the current project scope; dashed elements indicate planned extensions, not verified completed services.

event unless the application intentionally supports multiple categories. Venue and city fields should be stored separately so that location filters do not rely on parsing free-text descriptions. These are data-quality recommendations for the design

### C. Personalized Recommendation

Personalized recommendation is a planned intelligent extension, not a claim that a trained model is already deployed. The proposed baseline would consider event category, venue/location, budget, and date/time. Previous booking history can be included only when the application stores that history reliably. The candidate scoring formula in Eq. 1 defines a baseline for future implementation and testing.

### D. Agentic Assistance

The proposed assistant interprets natural-language requests, extracts constraints, retrieves matching events, and presents ranked options. After a user selects an event, it can guide the user through booking. The backend must validate availability, and explicit confirmation is required before booking or payment. The system must not report a booking as complete until the backend confirms it.

## V. METHODOLOGY

### A. Development Process

The methodology includes requirement analysis, architecture design, backend API development, frontend integration, recommendation design, and evaluation. Functional requirements include event search, event details, user management, recommendations, and booking management. Non-functional requirements include usability, maintainability, security, and response time.

### B. Backend and Frontend

Spring Boot is intended for backend service development, while REST APIs connect the React frontend to backend services. The API Gateway routes requests. User and event responsibilities should remain separate, and each service should own its data. React provides the interface for event browsing, filters, details, and booking workflows. Loading states, validation, and API error handling should reflect actual backend responses.

### C. Recommendation Model

Let $u$ denote a user and $e$ a candidate event. The proposed score is

$$R(u,e) = wcC(u,e) + wlL(u,e) + wbB(u,e) + wtT(u,e), \tag{1}$$

where $C$ is category similarity, $L$ is location compatibility, $B$ is budget compatibility, and $T$ is timing compatibility. The weights represent each attribute's importance and may satisfy

$$w_c + w_l + w_b + w_t = 1, \quad w_i \geq 0. \tag{2}$$

The scoring functions and weights must be defined and validated during implementation. This equation describes a proposed baseline, not a measured result.

Before ranking, hard constraints should be applied to exclude events that cannot satisfy the request. For example, an event outside the requested date range or over a strict maximum budget should not be promoted merely because its category score is high. Soft preferences can then be used to rank the remaining eligible events. This two-stage approach separates feasibility from preference: filtering answers whether an event is eligible, while ranking estimates how well an eligible event matches the user.

One possible normalization scheme maps each component to the interval $[0, 1]$. Category compatibility can be represented as an exact or partial category match. Location compatibility may use city equality or a distance-based function if reliable coordinates are available. Budget compatibility can score events more highly when their price is within the user's stated range. Timing compatibility can reward events that fall within the requested dates or preferred time window. The exact functions should be selected according to the data available and documented in the implementation.

The ranking process can be described in pseudocode as follows:

**Input:** user preferences $u$, event set $E$

**Output:** ordered eligible events $L$

1. Set $L$ to an empty list.

2. For each event $e$ in $E$, validate required event fields.

3. Reject $e$ if it violates a hard date, location, price, or availability constraint.

4. Compute $C(u,e)$, $L(u,e)$, $B(u,e)$, and $T(u,e)$.

5. Compute $R(u,e)$ using Eq. 1.

6. Append $(e, R(u,e))$ to $L$.

7. Sort $L$ by descending score and return the ranked list.

This pseudocode specifies the intended baseline only. Its actual behaviour depends on the implemented filters, attribute normalization, tie-breaking policy, and validation rules.

The formula is a conceptual specification rather than a claim of an already deployed model. No numerical scores are presented because recommendation experiments have not yet been reported.

### D. Agentic Workflow

The proposed agentic workflow includes interpreting a user's request, extracting event constraints, searching the available event listings, presenting options, and guiding the user through selection. Availability validation, persistent booking creation, and payment confirmation are future workflow steps unless verified in the current implementation. Any real reservation or payment action must require explicit user confirmation and a successful backend response.

An agentic interface should be treated as a controlled orchestration layer rather than as unrestricted authority to modify bookings. The assistant can interpret a request such as a preference for a music event in Hyderabad within a given budget, convert it into structured search constraints, and present matching event records. If the request omits an essential constraint, the system should ask a concise clarification rather than silently inventing a date or budget. The result should be grounded in the event catalogue returned by the backend.

The system should distinguish informational actions from consequential actions. Searching, filtering, and explaining why an event matches are low-risk operations. Creating a booking, changing ticket quantity, or initiating payment changes user state and therefore requires a clear confirmation step. A confirmation message should only be displayed after the backend reports success. If a request fails, the interface should report the failure and avoid presenting an unverified booking reference. This design principle is particularly important if an agentic assistant is added in a later implementation phase.

Natural-language interpretation also creates failure cases. A request may contain an ambiguous venue, a date that has already passed, an impossible budget, or conflicting preferences. The workflow should detect these cases and either request clarification or return no eligible results. It should not claim that tickets are available unless availability is checked by a reliable service. These safeguards form part of the proposed design and require dedicated tests before the workflow can be considered complete.

## VI. Functional and Non-Functional Requirements

### A. Functional Requirements

The core functional requirements are event listing, event detail retrieval, user-related operations, category-based discovery, and communication between the React frontend and Spring Boot APIs. Where endpoints exist, they should accept valid requests and return consistent responses. Invalid identifiers and malformed requests should return meaningful error statuses rather than unhandled server exceptions. Search and filtering requirements should be documented in terms of the fields actually available in the event data.

Recommendation functionality is a separate requirement because the existence of event listing APIs does not by itself establish personalization. The proposed recommender needs a defined input representation, eligibility filters, scoring functions, and deterministic ordering rules. Booking management likewise requires a persistent booking record, a relationship to a user and event, quantity and price validation, and a clear booking status. These capabilities should be marked as pending until the code and tests demonstrate them.

### *B. Non-Functional Requirements*

Usability requires consistent event cards, readable details, visible prices and dates, and understandable feedback for loading and error states. Maintainability benefits from separating user, event, and routing responsibilities. Reliability requires validation at both the frontend and backend, while security requires authentication and authorization before user-specific data or booking actions are exposed. Performance should be assessed under a stated test workload instead of inferred from a single successful request.

Availability and correctness are important in a ticketing application because event information can change. The interface should display a clear last-known status where appropriate and revalidate critical details before booking. Logging should record service errors without unnecessarily storing sensitive user information. These requirements provide a checklist for further development rather than a claim that all non-functional targets have already been met.

## VII. Data Flow and Service Responsibilities

The intended request path begins when the user interacts with the React interface. The frontend sends an HTTP request to the API Gateway, which routes the request to the appropriate backend service. Eureka provides service discovery for registered services in the Spring-based architecture. The Event Service is responsible for event-related operations, while the User Service handles user-related operations. The precise endpoints, ports, persistence configuration, and security settings must match the current source code and deployment environment.

Service boundaries are useful only when they are reflected in clear contracts. Each API should document its route, HTTP method, request fields, response structure, and error conditions. The frontend should not depend on internal database details. A service should validate incoming data even if the frontend has already validated it, because API requests can be made outside the browser interface. Integration tests should verify that gateway routing and service discovery work together rather than testing each component only in isolation.

A separate booking service and persistent booking database may be appropriate as the project grows, but they should not be drawn as completed components until they exist and are tested. If a demonstration uses a simulated payment amount or a mock confirmation, the paper and interface should explicitly identify it as a simulation. It must not be described as a real payment integration.

## VIII. Security, Privacy, and Responsible Automation

A ticketing platform can process names, contact information, preferences, and booking history. Only data required for the service should be collected, and access to account-specific information should be limited to authorized users. Authentication verifies identity, while authorization determines which operations that identity may perform. These controls should be enforced on the backend rather than relying only on hidden frontend buttons.

Recommendation history can reveal personal interests and attendance patterns. If previous bookings are used for personalization, users should be informed about the purpose of that use, and the application should avoid retaining data longer than necessary. Logs should exclude passwords, authentication tokens, and unnecessary personal details. Secrets such as database passwords should be stored in environment-specific configuration rather than committed to a public source repository.

Agentic automation should preserve user control. The assistant can propose events and explain ranking factors, but it should not complete a consequential booking or payment without explicit confirmation. The system should show the event, date, venue, quantity, total price, and relevant conditions before confirmation. This reduces the risk of an ambiguous natural-language request being converted into an unintended transaction.

## IX. Evaluation Methodology

A reproducible evaluation should begin with a fixed set of test cases. Functional tests can cover valid event retrieval, unknown event identifiers, category filters, missing fields, invalid dates, and backend service failures. Integration tests should send requests through the API Gateway and confirm that the correct service responds. Frontend tests should verify that successful responses are displayed correctly and that failures produce understandable messages.

Recommendation quality requires a labelled test set or user relevance judgments. For each test request, the expected relevant events should be recorded before computing Precision@K and Recall@K. A baseline such as category-only filtering can be compared with the contextual weighted score using the same queries and event catalogue. The evaluation should disclose how relevance was judged, how many queries were used, and whether the dataset represents real or synthetic events. Without these details, a numerical comparison would not be reproducible.

Performance tests should report the hardware or deployment environment, software configuration, request count, concurrency level, and measurement procedure. Response time may be reported using median and high-percentile values in addition to the mean, because averages can hide slow requests. Error rate should be calculated as failed requests divided by total requests under the defined workload. Results should be reported only after the tests are run; no numeric performance values are asserted in this paper.

## X. Limitations and Threats to Validity

The main limitation is that the proposed recommendation and agentic workflows may be ahead of the current implementation. A diagram or equation explains intended behaviour but cannot establish that the corresponding feature runs correctly. The current paper therefore distinguishes the service-based foundation from planned personalization and booking extensions. Any future revision should update this distinction after inspecting the repository and executing the relevant tests.

A second limitation concerns data quality. Incomplete event descriptions, inconsistent categories, missing prices, or outdated dates can reduce recommendation quality regardless of the ranking formula. A third limitation is the lack of supplied experimental results. Without a test dataset and measured outcomes, the paper cannot claim that EventGenie outperforms existing platforms or baselines. A fourth limitation is generalizability: an initial focus on Hyderabad and Indian live events may not represent the needs of users in other regions or event categories.

Future evaluation should address these limitations through a documented dataset, user relevance judgments, baseline comparisons, security testing, and performance experiments. Any claim of improved user satisfaction should be supported by a suitable user study rather than inferred from the presence of a recommendation interface.

## XI. Implementation and Evaluation Plan

The final paper must describe the features actually implemented in the EventGenie repository. Proposed components should not be represented as completed until tested.

### A. Functional Tests

Tests should first cover the services and endpoints that exist in the current repository, including event retrieval, user-related operations, invalid request handling, and frontend-to-API integration. Recommendation generation, booking creation, unavailable-event handling, and confirmation status should be tested after those features are implemented. This workflow is a proposed validation checklist, not evidence that every step currently works.

### B. Recommendation Metrics

When relevant labels or user feedback are available, Precision@K and Recall@K can evaluate recommendation quality. Precision@K measures the proportion of relevant events among the top $K$ results, while Recall@K measures the proportion of relevant events retrieved.

### C. Performance Metrics

The application should be tested under a documented workload. Response time, successful request rate, error rate, and booking-workflow correctness should be measured. Table II lists the proposed metrics; numerical results should be added only after actual experiments.

TABLE II
Proposed Evaluation Metrics

| Metric | Purpose |
|---|---|
| Test pass rate | Measures functional correctness across documented test cases. |
| Precision@K | Measures relevant events among the top $K$ recommendations. |
| Recall@K | Measures the proportion of relevant events retrieved. |
| Median and 95th-percentile latency | Measures typical and high-end response time under a stated workload. |
| Error rate | Measures failed requests divided by total requests. |
| Booking consistency | Checks that confirmed status matches a persisted backend record. |

## XII. Discussion

EventGenie aims to make event discovery more convenient by combining contextual ranking with guided booking. Location and timing are important because events have specific venues and limited lifecycles [3]–[5]. A content-based baseline can provide initial recommendations without extensive historical data, but quality depends on accurate metadata and meaningful preferences.

A modular architecture can improve maintainability but introduces service communication, data consistency, security, and deployment challenges. These require defined REST contracts, database ownership, authentication, authorization, and integration testing. The proposed agentic workflow must be evaluated for incorrect matches and unintended actions. Explicit confirmation should remain mandatory for consequential transactions.

The proposed design also emphasizes transparency. An explanation such as "shown because the selected category and date match" can help a user understand a recommendation, provided the explanation reflects the actual scoring logic. Explanations should not be generated independently of the data used to rank an event. If the system cannot verify a venue, price, or availability status, it should communicate that limitation rather than presenting uncertain information as fact.

The system's modular structure can support incremental development: first establish reliable event and user APIs, then integrate the frontend, and subsequently add ranking and guided booking. This sequence makes debugging easier because failures can be isolated by service boundary. However, microservices also introduce configuration and network dependencies; a local demonstration must verify that Eureka registration, gateway routing, and service URLs are configured consistently.

## XIII. Conclusion and Future Work

This paper presented EventGenie, an AI-Driven Agentic Smart Event Ticket Booking Platform intended for concerts, live bands, DJ performances, cultural festivals, and techno events. The proposed design combines a React frontend, Spring Boot backend services, contextual event ranking, and guided booking assistance.

The current project establishes the foundation for an event-focused discovery platform using a React frontend and Spring Boot services, including API routing, service discovery, user-related operations, and event-related operations. Contextual recommendation and the complete agentic booking workflow remain proposed extensions until implementation and testing confirm them. Future work includes validating all service endpoints, completing persistent booking management, implementing and evaluating recommendations, strengthening authentication and authorization, and collecting reproducible performance measurements. No performance graph or numerical recommendation result is claimed in this paper because actual experimental data have not been supplied.

The next development phase should prioritize automated tests for the currently available APIs, documentation of the event data schema, and verification of the complete frontend-to-backend request path. Once these foundations are stable, the contextual ranking baseline can be implemented and compared against a simple category-filtering baseline. A booking workflow can then be added with backend validation, persistent state, and explicit user confirmation. These steps would provide the evidence needed to move the work from a design proposal toward a validated intelligent event-booking system.

## REFERENCES


[1] Z. Qiao, P. Zhang, C. Zhou, Y. Cao, L. Guo, and Y. Zhang, "Event Recommendation in Event-Based Social Networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, no. 1, 2014. doi: 10.1609/aaai.v28i1.9095. [Online]. Available: https://doi.org/10.1609/aaai.v28i1.9095

[2] A. Arens-Volland and Y. Naudet, "Personalized recommender system for event attendees," in *Proceedings of the 11th International Workshop on Semantic and Social Media Adaptation and Personalization*, 2016, pp. 65–70. doi: 10.1109/SMAP.2016.7753386. [Online]. Available: https://doi.org/10.1109/SMAP.2016.7753386

[3] D. Horowitz, D. Contreras, and M. Salamó, "EventAware: A mobile recommender system for events," *Pattern Recognition Letters*, vol. 105, pp. 121–134, 2018. doi: 10.1016/j.patrec.2017.07.003. [Online]. Available: https://doi.org/10.1016/j.patrec.2017.07.003

[4] O. Jiménez, M. Salamó, and L. Boratto, "Distance-Aware Event Recommendation in Event-Based Social Networks," in *Artificial Intelligence Research and Development*, vol. 319, pp. 235–244. doi: 10.3233/FAIA190130. [Online]. Available: https://doi.org/10.3233/FAIA190130

[5] M. Wei and D. Wang, "CPERS: Contextual and Personalized Event Recommender System," in *Proceedings of the International Conference on Computational Science and Computational Intelligence*, 2016, pp. 421–426. doi: 10.1109/CSCI.2016.0086. [Online]. Available: https://doi.org/10.1109/CSCI.2016.0086

[6] G. Adomavicius and A. Tuzhilin, "Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, Jun. 2005. doi: 10.1109/TKDE.2005.99. [Online]. Available: https://doi.org/10.1109/TKDE.2005.99

[7] R. Burke, "Hybrid Recommender Systems: Survey and Experiments," *User Modeling and User-Adapted Interaction*, vol. 12, pp. 331–370, 2002. doi: 10.1023/A:1021240730564. [Online]. Available: https://doi.org/10.1023/A:1021240730564

[8] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021. [Online]. Available: https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/

[9] Spring, "Spring Boot Reference Documentation." [Online]. Available: https://docs.spring.io/spring-boot/index.html

[10] React, "React Documentation." [Online]. Available: https://react.dev/

[11] Spring, "Spring Cloud Netflix Reference Documentation." [Online]. Available: https://docs.spring.io/spring-cloud-netflix/reference/